

Day 2 — WHERE and Filtering

Today we move from:

"Give me data."

to:

"Give me only the data that matches a condition."

This is one of the most important SQL skills you'll use as a Data/GenAI engineer.

We'll work with a realistic RAG database.

1. Today's Goals

By the end of Day 2, you should be able to:

Use WHERE

Compare values with =, !=, <>, >, <, >=, <=

Filter text

Filter numbers

Filter dates

Combine conditions with AND

Combine conditions with OR

Use NOT

Understand operator precedence

Handle NULL correctly with IS NULL / IS NOT NULL

Debug filtering queries

Rule: You write the SQL first. I won't provide solutions until you attempt the exercises.

2. Our RAG Database

We'll use:

users

id	name	email	country	account_status
1	Aarav	aarav@example.com	India	active
2	Emma	emma@example.com	USA	active
3	Liam	liam@example.com	UK	inactive
4	Sofia	sofia@example.com	India	active
5	Noah	noah@example.com	USA	suspended

documents

id	user_id	title	file_type	page_count	status
101	1	RAG Fundamentals	pdf	20	processed
102	1	PostgreSQL Guide	pdf	45	processed
103	2	Python Basics	pdf	30	processing
104	3	FastAPI Tutorial	md	15	failed
105	4	Vector Databases	pdf	50	processed
106	5	Docker Guide	pdf	25	processing

3. Basic WHERE

The general structure:

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example:

```
SELECT
id,
name
FROM users
WHERE country = 'India';
```

This means:

Return users whose country is India.

Result:

id	name
1	Aarav
4	Sofia

4. WHERE Works Row by Row

Imagine:

```
SELECT *
FROM users
WHERE country = 'India';
```

Conceptually PostgreSQL evaluates each row:

Aarav → India → TRUE
Emma → USA → FALSE
Liam → UK → FALSE
Sofia → India → TRUE
Noah → USA → FALSE

Only rows where the condition evaluates to TRUE are returned.
This mental model is extremely useful for debugging.

5. Comparison Operators

SQL provides:

Operator	Meaning
=	equal
!=	not equal
<>	not equal
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal

Equality

```
SELECT *  
FROM documents  
WHERE file_type = 'pdf';
```

Returns only PDF documents.

Greater Than

```
SELECT  
title,  
page_count  
FROM documents  
WHERE page_count > 30;
```

This finds documents containing more than 30 pages.

Greater Than or Equal

```
SELECT  
title,  
page_count  
  FROM documents  
 WHERE page_count >= 30;
```

Notice the difference:

> 30

doesn't include 30.

>= 30

does.

Less Than

```
SELECT  
title,  
page_count  
  FROM documents  
 WHERE page_count < 30;
```

Not Equal

You can use:

```
WHERE status != 'processed'
```

or:

```
WHERE status <> 'processed'
```

Both mean "not equal" in PostgreSQL.

6. Text Values Need Quotes

Correct:

```
WHERE country = 'India'
```

Incorrect:

```
WHERE country = India
```

Why?

India without quotes is interpreted as an identifier rather than a string value.

String:

'India'

Column:

Country

7. Numbers Don't Need Quotes

Correct:

```
WHERE page_count > 30
```

Generally don't write:

```
WHERE page_count > '30'
```

Use the appropriate data type.

Think:

'India' → text

30 → number

8. Multiple Conditions — AND

Suppose we want:

PDF documents with more than 30 pages.

SELECT

```
title,  
file_type,  
page_count  
    FROM documents  
    WHERE file_type = 'pdf'  
AND page_count > 30;
```

Both conditions must be true.

Conceptually:

PDF? Pages > 30? Result

```
TRUE TRUE TRUE  
TRUE FALSE FALSE  
FALSE TRUE FALSE  
FALSE FALSE FALSE
```

9. Multiple Conditions — OR

Suppose we want:

Documents that are either PDF or Markdown.

```
SELECT  
title,  
file_type  
    FROM documents  
    WHERE file_type = 'pdf'  
OR file_type = 'md';
```

Only one condition needs to be true.

10. AND vs OR

This distinction causes many SQL bugs.

AND

```
WHERE country = 'India'
AND account_status = 'active'
```

Means:

India AND active.

OR

```
WHERE country = 'India'
OR account_status = 'active'
```

Means:

India OR active.

The second query can return significantly more rows.

11. Combining AND and OR

Suppose you want:

Active users from India OR the USA.

Don't rely on complicated implicit logic.

Use parentheses:

```
SELECT
id,
name,
country
FROM users
WHERE account_status = 'active'
AND (
country = 'India'
OR country = 'USA'
);
```

Parentheses make your intended logic explicit.

12. Operator Precedence

Consider:

```
WHERE country = 'India'  
OR country = 'USA'  
AND account_status = 'active'
```

SQL evaluates AND before OR.

So conceptually this becomes:

```
country = India  
OR  
(country = USA AND account_status = active)
```

That may not be what you intended.

If you mean:

```
(India OR USA)  
AND active
```

write:

```
WHERE (  
country = 'India'  
OR country = 'USA'  
)  
AND account_status = 'active';  
Engineering rule
```

When mixing AND and OR:

Use parentheses to make the business logic obvious.

13. NOT

You can negate a condition.

```
SELECT *  
FROM users  
WHERE NOT country = 'India';
```

You can also use:

```
WHERE country <> 'India';
```

For simple comparisons, the latter is usually clearer.

14. NULL — Very Important

Suppose:

documents.status

can be NULL.

You might think this works:

```
WHERE status = NULL
```

It does not correctly test for NULL.

Use:

```
WHERE status IS NULL;
```

And:

```
WHERE status IS NOT NULL;
```

Remember

status = NULL

status IS NULL

15. Why NULL Is Special

SQL uses three-valued logic:

TRUE

FALSE

UNKNOWN

For example:

`page_count = NULL`

doesn't produce normal TRUE/FALSE comparison behavior.

NULL means the value isn't known/present.

Therefore:

`IS NULL`

is specifically designed for checking it.

We'll revisit this repeatedly because NULL-related bugs are common in production SQL.

16. Real RAG Engineering Example

Imagine your document ingestion pipeline has statuses:

processing

processed

failed

An API endpoint needs:

"Give me all documents that still need attention."

You might define "needs attention" as:

processing OR failed

The SQL would conceptually require:

```
WHERE status = 'processing'  
OR status = 'failed'
```

This type of filtering is common in:

document ingestion pipelines

background workers

retry queues

RAG indexing

monitoring dashboards

17. Filtering by Numeric Data

Suppose you want large documents:

```
SELECT  
id,  
title,  
page_count  
FROM documents  
WHERE page_count >= 40;
```

Or small documents:

```
WHERE page_count < 20;
```

18. Filtering by Status

Example:

```
SELECT  
id,  
title  
FROM documents  
WHERE status = 'failed';
```

This could be used by an ingestion-monitoring service to find failed documents.

19. Filtering Users

Example:

```
SELECT
id,
name,
email
    FROM users
    WHERE account_status = 'active';
```

Then combine:

```
SELECT
id,
name,
email
    FROM users
    WHERE account_status = 'active'
AND country = 'India';
```

20. Debugging Challenge #1

A developer writes:

```
SELECT *
FROM documents
WHERE file_type = pdf;
```

PostgreSQL reports an error.

Your task

Explain:

What's wrong?

How would you fix it?

Why does PostgreSQL interpret pdf incorrectly?

Don't ask me for the answer yet.

21. Debugging Challenge #2

Developer writes:

```
SELECT *  
FROM documents  
WHERE status = NULL;
```

They expect all documents where status is missing.

Your task

Is this correct?

If not, write the corrected query.

Explain why.

22. Exercise Set A — Basic Filtering

Using the tables above:

Q1

Return all users from India.

Q2

Return all documents where file_type is pdf.

Q3

Return all documents with more than 30 pages.

Q4

Return all documents with 30 or fewer pages.

Q5

Return all users whose account_status is not active.

23. Exercise Set B — AND

Q6

Find all PDF documents with more than 30 pages.

Return:

id
title
page_count

Q7

Find active users from India.

Return:

id
name
email

Q8

Find processed PDF documents.

Return:

id
title
file_type
status

24. Exercise Set C — OR

Q9

Find documents whose status is either:

processing

failed

Q10

Find users who are either:

India

USA

Q11

Find documents that are either:

PDF with more than 40 pages

OR

Markdown files

Be careful with your parentheses.

25. Practical Engineering Problem

You're building a RAG document-processing dashboard.

The dashboard needs to show documents that require worker attention.

Business rule:

Show documents whose status is processing OR failed, but only if they contain at least 20 pages.

Return:

document_id

title

status

page_count

Your task

Write one SQL query.

Think carefully about:

AND

OR

()

26. Debug This Production Query

A developer wants:

Active users from India or USA.

They wrote:

SELECT

id,

name,

country

FROM users

WHERE country = 'India'

OR country = 'USA'

AND account_status = 'active';

Tasks

1. What logical condition does PostgreSQL actually evaluate?

2. Why can this return incorrect users?

3. Rewrite the query so it exactly matches the business requirement.

27. Interview Question

Answer as if you're in a real interview:

What is the difference between WHERE and SELECT?

Then answer:

Why can't we use status = NULL to find NULL values?

28. Explanation Challenge

Explain this query in plain English:

```
SELECT
id,
title,
page_count
  FROM documents
 WHERE file_type = 'pdf'
AND page_count > 30
AND status <> 'failed';
```

Don't just translate each line.

Explain what records the query is trying to find and why.

29. Day 2 Short Test

Attempt without looking at the explanation.

Q1

What does WHERE do?

Q2

Write a query to find users from India.

Q3

Write a query to find documents with more than 25 pages.

Q4

What's the difference between:

=

and:

<>

Q5

What's wrong with:

```
WHERE status = NULL
```

Q6

Write the correct condition for:

status is NULL.

Q7

What is the difference between AND and OR?

Q8

Why are parentheses important here?

```
WHERE A OR B AND C
```

Q9

Write a query to find:

active users from India.

Q10

Write a query to find:

PDF documents with at least 40 pages that are not failed.